

Benchmark Report: pyflate

A small link budget, a compressed payload, and a lot of work between the bits

Matan Cohen · Yuval Kogan | Technion | HWSW Final Project

Somewhere between a spacecraft and a ground station there is a link budget measured in kilobits.

Compression makes the most of it; decompression turns the arriving bits back into useful data. This benchmark gives that quiet half of the job to Python: read a bit stream, recover symbols, and undo several transformations. Its Huffman matcher looks like the obvious place to start. Measuring it led us to improvements across the whole decoding pipeline, then a Rust extension that executes the symbol decoder through one call per block.

1. Workload and result

Despite its name, the measured pyflate workload is **bzip2**, not DEFLATE. Each iteration decompresses `interpreter.tar.bz2` from **67,562 bytes to 399,360 bytes**; the harness checks the original MD5 outside the timer. The Python tier uses the standard library rather than calling a native decompression library. Its main structures are an integer bit buffer, Huffman tables, a move-to-front (MTF) alphabet, Burrows-Wheeler transform (BWT) index vectors and output buffers.

One block, six Huffman tables, 148,271 symbols. The decoder switches tables every 50 symbols. Huffman decoding, move-to-front and RUNA/RUNB expansion produce a 336,184-byte L-vector. Inverse BWT and final RLE4 expansion produce the 399,360-byte output.

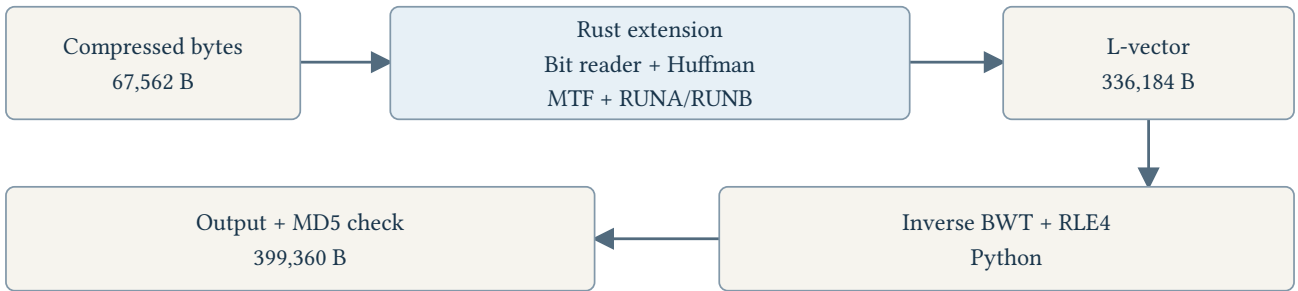


Figure 1: The bzip2 path. RUNA/RUNB and move-to-front are handled together in the symbol loop. The Rust kernel replaces that boundary in one call per block, and it is the same boundary the two accelerators implement.

Configuration	Mean ± SD	vs stock
Stock Python	1,123.49 ± 10.99 ms	1.00×
Optimized Python	281.16 ± 3.05 ms	4.00×
Python + Rust extension	170.01 ± 2.30 ms	6.61×

Our final software path is 6.61× faster than stock: an 84.9% runtime reduction. Python improvements provide 4.00×; over the same benchmark’s Python back end, the Rust symbol decoder adds **1.67×** (Section 4). Both tiers exceed the course’s 7% improvement requirement and match `bz2.decompress` byte for byte, passing the unchanged MD5 check. Rust handles the blue stage above; block headers, inverse BWT and RLE4 remain in Python.

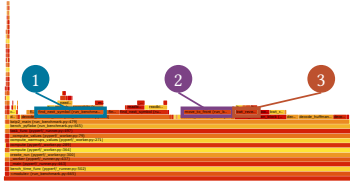
Measurement scope. Course QEMU/KVM VM, Ubuntu 22.04, release CPython 3.10.12; 120 values per configuration, pinned to guest CPU 0, from revision 2c8c754 through the documented scripts and its own Rust build. Source: `results/vm_canonical_20260910_2c8c754/suite/` (Appendix A7). Development profiles and ablations are labeled separately.

Medians are 1,122.68, 280.43 and 169.67 ms, with interquartile ranges of 1.2-1.3% of the median. Worker grouping is milder than nbody’s (ICC 0.48, 0.27 and 0.19), so the 120 values are worth about 61 to 87 independent observations; Appendix A5 gives the decomposition.

2. Profiling: the whole decoding pipeline

The stock Python-frame profile highlights Huffman decoding and bit-buffer work, with move-to-front and inverse BWT alongside them. Development instrumentation found a mean of about five entries visited by the symbol matcher, despite its table capacity of 258. A lookup table can remove this scan, but the measured scan is short; per-symbol interpreter work and the later transformations also deserve attention.

Full recorded flame graph

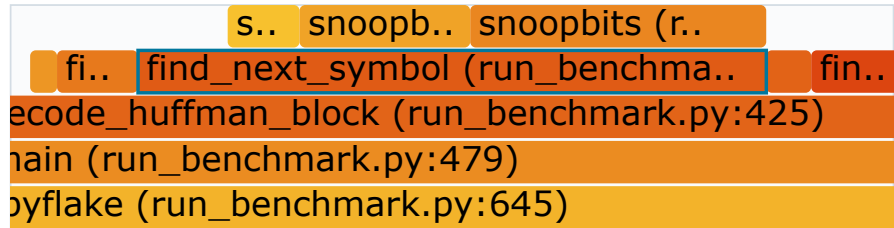


Original widths and stack depth.
Imports and harness retained.
Each outline marks one call path.
py-spy emits one frame per
source line, so a function is
many boxes; shares are totals.

Highlighted regions - enlarged with surrounding frames

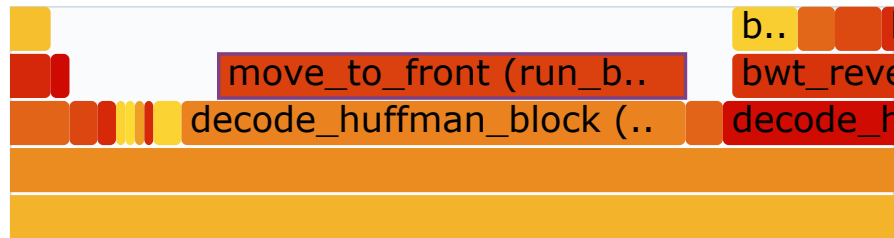
1. Huffman matcher

find_next_symbol | 40.87% inclusive, across 7 per-line frames; widest 18.82% outlined



2. Alphabet updates

move_to_front | 13.44% inclusive



3. Inverse BWT

bwt_reverse | 14.52% inclusive, across 5 per-line frames; widest 6.18% outlined

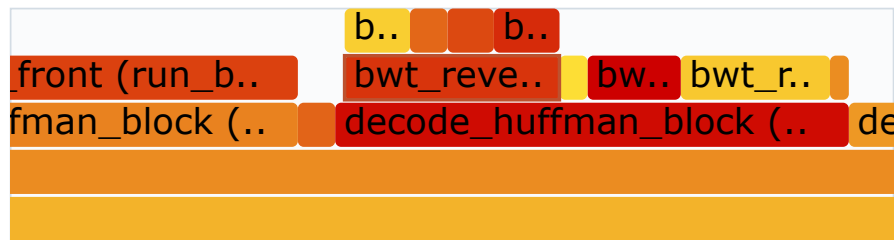


Figure 2: Full stock Python-frame flame graph, retaining startup and harness context. Numbered outlines identify exactly the call paths enlarged on the right. Inclusive percentages use the original whole-profile denominator; nested shares overlap.

Function	Stock self	Stock incl.	Opt. self	Opt. incl.
decode_huffman_block	26.08%	97.31%	1.64%	82.79%
_decode_symbols_python	–	–	32.79%	32.79%
find_next_symbol	12.63%	40.86%	–	–
move_to_front	13.44%	13.44%	0.82%	0.82%
snoopbits	9.41%	16.13%	–	–
readbits	9.95%	13.44%	2.46%	2.46%
_mask	5.91%	5.91%	–	–
bwt_reverse	8.33%	14.52%	13.11%	38.52%
bwt_transform	6.18%	6.18%	25.41%	25.41%
rle4_expand	–	–	5.74%	5.74%

Self is time in the function itself; **inclusive** is time in it and everything it calls. Shares are of each profile's own total and are not comparable between the two columns, which are independently normalized – the optimized run is 4.00× shorter in absolute time. A dash means the function does not appear in that profile. The rewrite folded the stock bit reader and symbol matcher into one symbol-decode loop, `_decode_symbols_python` – the Python back end the Rust kernel replaces – and inlined its move-to-front as

`l.append(l.pop(-r))`. The few readbits and `move_to_front` samples left in the optimized profile come from block-header selector parsing in `compute_selectors_list`.

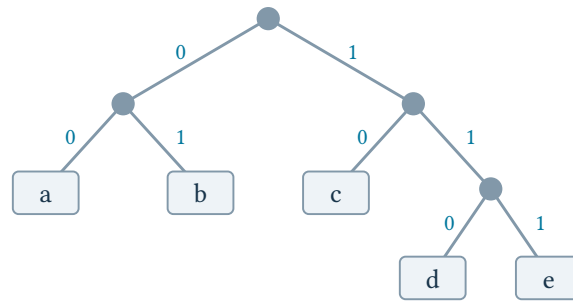
py-spy emits one frame per source line, so a function appears as several boxes side by side. The figure sums them, which is why its `find_next_symbol` label agrees with this table's 40.86% inclusive rather than reporting the 18.82% of the single widest box it outlines. Regenerate both from the same recorded SVGs the figures use with `report/summarize_profiles.py`.

The table is the argument for what shipped. In the stock profile the symbol loop and its bit reader account for the bulk of the work, and the inverse BWT (14.52% inclusive) is a minor share by comparison. After the rewrite, symbol decoding is still the largest single function (`_decode_symbols_python`, 32.79% self), but the inverse BWT now matches it: `bwt_reverse` holds 38.52% inclusive, most of it in `bwt_transform`. That is why the Rust kernel is drawn around the symbol decoder, and why the remaining Amdahl ceiling is the BWT.

A separate **Windows / CPython 3.12.6** cProfile run gives `find_next_symbol` 0.519 s cumulative out of 1.186 s (43.8%), including its bit-reader callees. `decode_huffman_block` has 0.241 s **self** time and 1.172 s **cumulative** time: most of its inclusive cost is in functions it calls. Source: `dev/pyflate/FINDINGS.md §3`. That development run and the VM profile above agree on the ordering, not on the numbers; different machine, different interpreter, different estimator.

Example codes

a = 00
b = 01
c = 10
d = 110
e = 111



Canonical ranges

len 2: 00..10

len 3: 110..111

Match a length range,
then index the symbol.

Example: 110 decodes to d.

Figure 3: Illustrative canonical Huffman codes. Codes of equal length are consecutive; length-indexed ranges identify the symbol without scanning individual candidates. The optimized implementation adds a primary lookup for short codes.

3. Changes that shipped

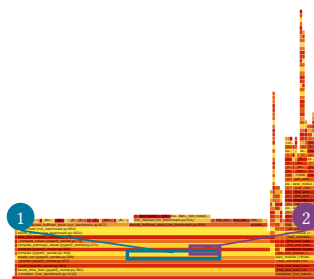
- **Reduce per-byte overhead:** read the compressed input once, refill the bit window in chunks, bind hot values to locals and accumulate bytes without allocating an object per byte.
- **Decode canonical Huffman codes:** use length-indexed tables and a primary lookup table with a fallback for longer codes. Table construction is amortized over the block.
- **Improve the back end:** replace comparison sorting with counting-sort BWT construction; reverse the MTF list so updates move $O(\text{rank})$ entries; use regex-assisted RLE4 expansion to reduce Python loop iterations. The byte scan/output work is still linear overall.

Component reverted from T3	VM trial 1	VM trial 2	Dev, 3.12
None (full T3 runtime)	271.2 ms	273.9 ms	175.5 ms
Regex-assisted RLE4	+100.6 ms	+98.7 ms	+32.3 ms
Primary Huffman lookup	+52.6 ms	+49.4 ms	+10.1 ms
Counting-sort BWT	+20.8 ms	+17.2 ms	+38.6 ms

Ablation scope. Each row puts one stock component back into the optimized Python decoder (dev/pyflate/ablate.py, which drives the T3 module directly rather than the pyperf harness) and reports the added time, best of seven interleaved decompressions. Costs need not add. The two VM trials are separate runs on the course VM, release CPython 3.10.12, pinned to one guest CPU; the last column is the earlier development run on Windows / CPython 3.12.6 (dev/pyflate/FINDINGS.md §2b).

The measured platform reverses part of the development ranking. On the VM, regex-assisted RLE4 is worth about 100 ms and the primary Huffman lookup about 50 ms – both more than counting-sort BWT at about 20 ms – and the two trials agree within 4 ms on every row. The development run on 3.12 had ranked BWT first and the Huffman table last, so the ordering is a property of the interpreter and platform, not of the algorithms. What survives on both is that no single change dominates: the pipeline, not the matcher alone, had to be optimized. Raw output: results/vm_rerun_20260910_3697a63/.

Full recorded flame graph

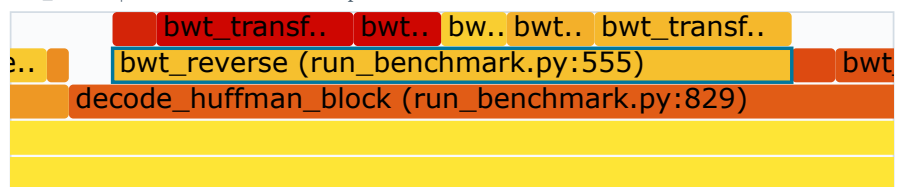


Original widths and stack depth.
Imports and harness retained.
Each outline marks one call path.
py-spy emits one frame per
source line, so a function is
many boxes; shares are totals.

Highlighted regions - enlarged with surrounding frames

1. Inverse BWT

bwt_reverse | 38.53% inclusive, across 4 per-line frames; widest 25.41% outlined



2. Index-table construction

bwt_transform | 25.42% inclusive, across 6 per-line frames; widest 7.38% outlined

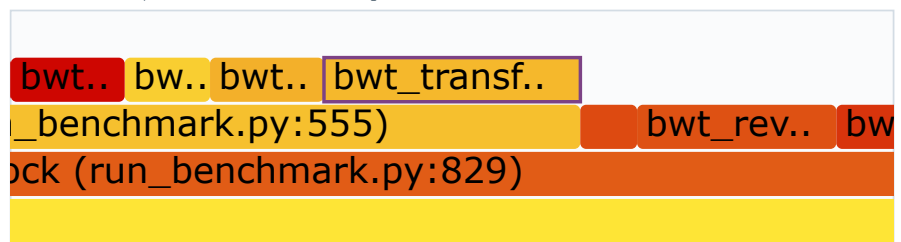


Figure 4: Full optimized **Python** profile, before native offload. Inverse BWT and its index-table construction remain visible beside `_decode_symbols_python`, the single symbol-decode loop that replaced the stock matcher and bit-reader frames.

4. Native execution: what improves, what remains

Our Rust/PyO3 BlockDecoder handles the bit reader, Huffman decode, MTF and RUNA/RUNB expansion. Header parsing, inverse BWT, RLE4 and MD5 remain in Python. One block-level call returns the L-vector and the ending bit position; checking both catches output errors and stream desynchronization.

Same benchmark file	Mean $\pm$ SD	vs Python
Python backend	283.88 $\pm$ 3.34 ms	1.00×
Native decode backend	170.01 $\pm$ 2.30 ms	1.67×

The native row is the same measurement as the Python + Rust row on page 1. The Python row runs the same benchmark file with its Python back end directly under pyperf, not through pyperformance, so it differs slightly from page 1's optimized Python; both cover the complete benchmark. `HWSW_BACKEND=python|native|auto` selects the implementation; native mode requires the extension, while auto mode falls back. The benchmark forwards the variable to pyperf's workers itself, so a scrubbed worker environment cannot quietly change which back end is measured.

The submitted Rust source was rebuilt on the VM. Eleven crate tests and seven blocks across five Python/bz2 fixtures passed, including exact ending bit positions; the eleventh is a property test that decodes random Kraft-complete Huffman tables against an independently written canonical encoder. A dispatch check confirmed that native and auto load the new extension and call it once per block.

Matched-work CPU counters

These are medians of three warm-loop VM runs, each decoding the input 16 times, divided by 16. Both backends produced the same output digest. They are a separate experiment from the rigorous timing table above, with acknowledged counter gating after setup (Appendix A2).

Metric per complete decode	Python	Hybrid native
Elapsed time	288.68 ms	174.44 ms
Instructions	1,947.94 M	1,171.65 M
Cycles	675.92 M	404.24 M
IPC	2.88	2.90
Branches	349.43 M	207.30 M
Branch misses (rate)	544,260 (0.156%)	301,458 (0.145%)
Generic cache references	1.749 M	1.686 M
Generic cache misses (rate)	69,322 (3.964%)	52,842 (3.133%)

Native decoding reduces instructions by **39.9%**, cycles by **40.2%** and branch misses by **44.6%**, while IPC changes little. The approximately 1.65× speedup in this experiment comes mainly from removing interpreter work. Generic cache misses fall **23.8%**, a smaller reduction than instructions: the remaining Python BWT/RLE4 pipeline still processes the whole block. Aggregate counters cannot attribute misses to a particular stage. L1 events are omitted because an earlier capture returned invalid load counts.

Offload limits and the next target

Amdahl's law gives $S = 1 / ((1 - f) + f/s)$ for kernel speedup s and offloaded fraction f of **optimized Python** runtime. Even infinite symbol-decoder speed leaves BWT and RLE4. Unlike nbody's native integration loop, this offload leaves over a billion instructions per decode in a largely Python pipeline. The next target is the **whole inverse BWT**, including table construction, not just its dependent traversal. Earlier VM phase timings put them at 137.7 ms and 47.8 ms respectively (Appendix A3). A native port and working-set sweep would test whether memory latency then becomes limiting; the present data do not establish that.

5. Hardware acceleration: measured

`huffman_engine` followed by `mtf_cam` produces the same L-vector as the Rust kernel, over the same boundary. Software keeps block headers, inverse BWT, RLE4 and MD5. Both modules are implemented in SystemVerilog, verified, and taken through synthesis and place-and-route; the numbers below are measured, not targets.

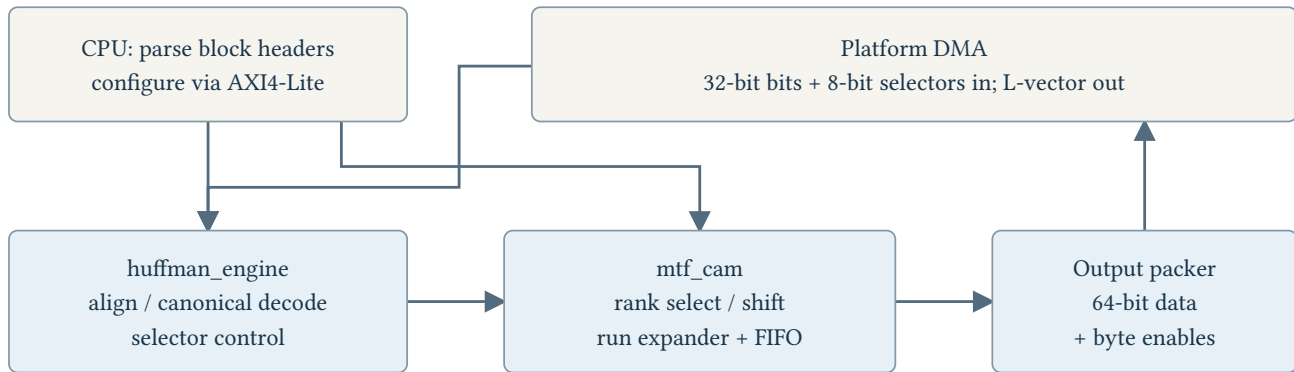


Figure 5: AXI4-Lite configures both modules; AXI4-Stream carries data. The symbol stream stays on chip. Platform DMA returns the L-vector to software.

Huffman engine

A bit aligner feeds a canonical-code comparator/lookup datapath; a selector controller changes the active table every 50 symbols. Inputs are a 32-bit compressed stream and an 8-bit selector stream; outputs are 32-bit beats carrying a 9-bit symbol plus type fields. A 32-bit AXI4-Lite window holds controls, `START_BIT`, lengths and counters, for six tables and bzip2's 20-bit maximum code length. It decodes the benchmark block in **149,276 cycles** for 148,271 symbols, so **K1 = 1.0068 cycles per symbol** including table construction and switches, and is trace-exact against the golden model over every beat.

MTF and run expansion

The input is a **rank**, so the decoder indexes the alphabet, shifts the preceding entries and expands RUNA/RUNB groups through a buffered output packer at eight bytes wide. It sustains **K3 = 1.0686 cycles per symbol**, and the CAM is 68% of its area.

Measured cost and what it buys

sky130, Yosys + OpenLane	<code>huffman_engine</code>	<code>mtf_cam</code>
Cells	151,058	18,814
Area	1.634 mm ²	0.187 mm ²
Fmax	≈ 8.9 MHz	≈ 37.6 MHz
Power	not reported	≈ 13.7 mW
Cycles per symbol	1.0068	1.0686
Directed + random tests	17 / 17	16 / 16
Line / toggle coverage	90.4% / 90.3%	92.0% / 93.8%

The two clocks are not measured the same way and must not be compared. `mtf_cam` closed post-CTS static timing at 37.6 MHz; `huffman_engine` is a pre-place-and-route estimate, because its placement did not converge, and its real figure is likely lower. Neither module reaches the 50 MHz design target, and for this workload that barely matters: the software residual dominates.

Amdahl decides the end-to-end gain, not the clock. Huffman decoding is 49.6% of the stock benchmark and move-to-front a further 13.4%. Offloading Huffman alone gives **1.97×** at 50 MHz and **1.93×** at the measured 8.9 MHz — a 5.6× clock difference worth 0.04× end to end, because 3 ms of hardware is negligible against 570 ms of surviving software. `mtf_cam` alone gives **1.15×**, which is a **25× speedup of its own stage** seen through a 13.4% slice. Chained on chip the two remove $f \approx 0.63$ of the benchmark, an ideal ceiling of $\approx 2.7\times$.

Trade-offs. `huffman_engine` is storage-dominated: a 17.3 kbit symbol table and an 8.6 kbit length window are flip-flops, because the open sky130 flow has no SRAM compiler. Mapping both to SRAM

macros is the one change that would bring it under 1 mm², and it would cost nothing in cycles per symbol. `mtf_cam`'s output width was swept at $W = 4, 8$ and 16: $W = 16$ buys 0.040 cycles per symbol for 11% more area and is not worth it, so $W = 8$ ships. Formal proofs bound the MTF invariants to depth 24 on the 256-entry list rather than exhaustively; that limit is recorded, not papered over.

6. Conclusion

The delivered Python-plus-Rust path decompresses the input in **170.01 ms**, achieving **6.61× over stock** with byte-exact output. Python improvements provide 4.00× and the native decoder adds 1.67×, leaving BWT and RLE4 as the next software targets.

Both accelerators are implemented and measured rather than proposed. The gain they buy is bounded by what stays in software: the inverse BWT and RLE4, which is the same ceiling the native decoder ran into.

Evidence: `hw/huffman_engine/docs/{ppa,integration}.md`, `hw/mtf_cam/docs/{ppa,integration}.md`.

Methods and profiles: **report_appendix.pdf**. Repository: [HWSW_Project](#).